ABSTRACT

The Parking Slot Management System is a data structure–based project designed to simulate the operation of a linear parking lot using a stack. In this system, cars are parked and removed following the Last-In-First-Out (LIFO) principle, which makes the stack an ideal structure for modeling real-life scenarios where vehicles enter and exit through a single lane.

The project enables key operations such as adding a car (push), removing a specific car (pop using a temporary stack), and displaying the current parking status. When a car positioned deeper in the stack needs to be removed, the cars above it are temporarily transferred to an auxiliary stack and then restored after removal, ensuring proper order.

This project demonstrates efficient stack operations, manipulation of linear data, and real-world applications of abstract data types. It also highlights how data structures can be applied to solve practical management problems in an organized and computationally effective manner.

INTRODUCTION

The Parking Slot Management System is a data structure–based project designed to simulate the operation of a linear parking lot using a stack. In this system, cars are parked and removed following the Last-In-First-Out (LIFO) principle, which makes the stack an ideal structure for modeling real-life scenarios where vehicles enter and exit through a single lane. The project enables key operations such as adding a car (push), removing a specific car (pop using a temporary stack), and displaying the current parking status. When a car positioned deeper in the stack needs to be removed, the cars above it are temporarily transferred to an auxiliary stack and then restored after removal, ensuring proper order.

- This project is based on the **Parking Slot Management System** implemented using the **Stack (LIFO) data structure**.
- It simulates real-life narrow parking spaces where vehicles can move only in a **last-in-first-out** manner.
- The system uses basic stack operations such as **push (park a car)** and **pop (remove a car)**.
- It helps maintain organized records of vehicles entering and exiting the parking area.
- The project shows how **data structures can be applied to real-world problems** for efficient resource management.
- It enhances understanding of stack functionality through a practical and interactive example.
- The system is simple, easy to implement, and demonstrates clear use of logical programming and data management.

This project demonstrates efficient stack operations, manipulation of linear data, and real-world applications of abstract data types. It also highlights how data structures can be applied to solve practical management problems in an organized and computationally effective manner.

EXISTING SYSTEM AND PROPOSED SYSTEM

➤ **Manual Parking Management**

- Parking attendants manage vehicle entry and exit manually.
- No automated tools or structured method is used.

➤ **Lack of Organized Structure**

- Cars are parked randomly without any defined order.
- Difficult to rearrange cars when one in the middle needs to leave.

➤ **High Chances of Errors**

- Human errors occur in recording vehicle numbers and slot usage.
- Miscommunication leads to misplaced or untracked vehicles.

➤ **Time-Consuming Process**

- Removing a car often requires shifting multiple vehicles.
- Entry/exit management becomes slow during peak hours.

➤ **Inefficient Record Keeping**

- Details are maintained on paper or verbally, leading to data loss.
- Hard to find specific vehicle information quickly.

➤ **Poor Utilization of Parking Space**

- Slots are not efficiently used due to lack of systematic arrangement.
- Results in overcrowding or unused spaces.

PROPOSED SYSTEM

➤ **Automated Parking Management Using Stack**

- Introduces a computer-based system using the **LIFO (Last-In-First-Out)** principle.
- Uses stack operations to efficiently manage parking slots.

➤ **Structured Vehicle Arrangement**

- Cars are parked in a sequential stack order to ensure easy entry and exit.
- Prevents random parking and confusion during car removal.

➤ **Fast and Error-Free Operations**

- Minimizes human error by automating record tracking.
- Quick identification of parked or removed vehicles.

➤ **Efficient Use of Parking Space**

- Slots are used in a disciplined and organized manner.
- Reduces overcrowding and empty unused spaces.

➤ **Simple Push and Pop Mechanism**

- **PUSH:** Car enters and is added to the top of the stack.
- **POP:** The last entered car is removed quickly without searching.

➤ **Improved Vehicle Tracking**

- Maintains a real-time record of all parked vehicles.
- Instant information retrieval when needed.

➤ **User-Friendly and Easy to Implement**

- Requires minimal resources and simple data structure logic.
- Suitable for small to medium parking areas.

SOFTWARE AND HARDWARE REQUIREMENTS

SOFTWARE REQUIREMENTS

➤ **Operating System**

- Windows 7/8/10/11
- Linux / Ubuntu (optional)
- macOS (optional)

➤ **Programming Language**

- C / C++ / Java / Python (any one used for implementation)

➤ **Development Tools / IDE**

- Code::Blocks / Dev C++ (for C/C++)
- Eclipse / IntelliJ IDEA / NetBeans (for Java)
- PyCharm / VS Code (for Python)
- Any text editor for coding

📌 HARDWARE REQUIREMENTS

➤ **Processor**

- Intel Core i3 or above
- AMD equivalent

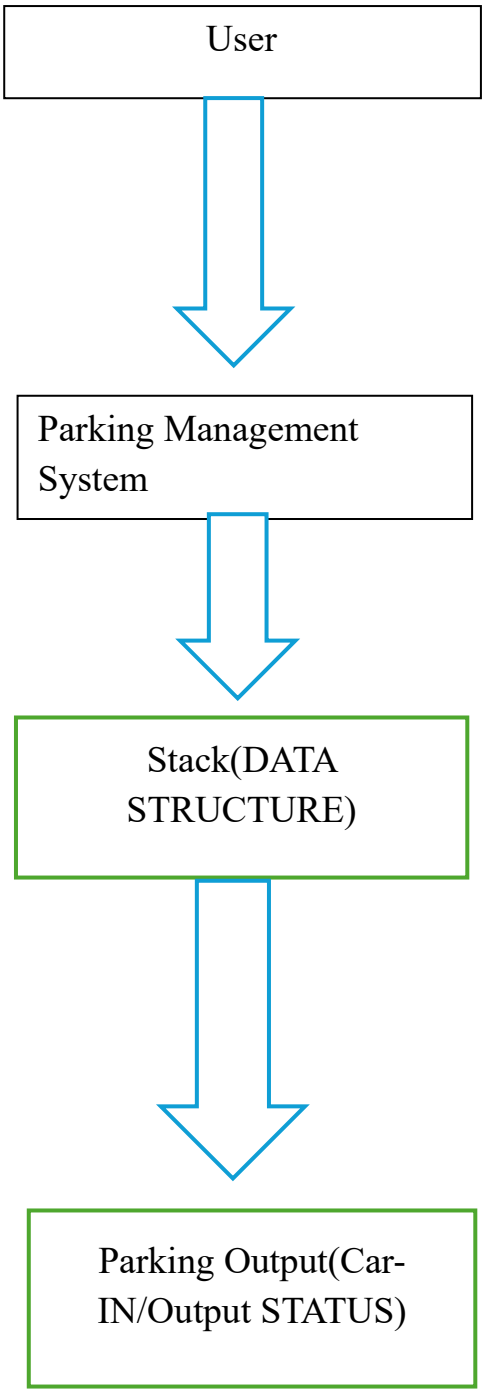
➤ **RAM**

- Minimum 2 GB
- Recommended: 4 GB or higher

➤ **Storage**

- Minimum 500 MB free space
- Extra space for project files and document.

ARCHITECTURAL DIAGRAM



METHODOLOGY

)

🔹 Step 1: Identify the Problem

- Understand issues in the current parking system.
- Notice delays, confusion, and difficulty in managing vehicles.

🔹 Step 2: Gather Requirements

- List the software and hardware needed.
- Decide what the system should do: park, remove, and display cars.

🔹 Step 3: Design the System

- Plan how the system will work.
- Draw simple diagrams (flowchart/architecture).
- Choose stack as the main structure.

🔹 Step 4: Select Data Structure

- Use **Stack (LIFO)** to manage parking slots.
- Define basic operations like push and pop.

🔹 Step 5: Develop Algorithms

- Write simple steps for parking, removing, and showing cars.
- Ensure the logic is correct and easy to implement.

RESULTS

Parking Slot Management (Stack)

1. Park a Car
2. Remove a Car
3. Display Parking
4. Exit

Enter your choice: 1

Enter Car Number: 101

Car 101 parked successfully.

Enter your choice: 1

Enter Car Number: 202

Car 202 parked successfully.

Enter your choice: 3

Current Cars in Parking:

202 101

Enter your choice: 2

Car 202 removed.

Enter your choice: 3

Current Cars in Parking:

101

CONCLUSION

The Parking Slot Management System using the Stack data structure provides an effective and organized way to manage vehicle parking in a limited and linear space. By applying the LIFO principle, the system simplifies the process of parking and removing vehicles while reducing confusion and human errors often seen in manual methods.

This project demonstrates how fundamental data structures can be used to solve real-world problems through logical design and automation.

The system is simple, reliable, and easy to implement, making it suitable for small parking areas or educational purposes. Overall, the project successfully shows the importance of structured data handling and highlights how technology can improve everyday operations.

FUTURE ENHANCEMENT

1. IoT Integration
    - Integrate with IoT devices for real-time monitoring and automated slot detection
    - Use sensors and cameras to track vehicle entry and exit
2. AI-powered License Plate Recognition
    - Implement AI-driven license plate recognition for automated vehicle identification
    - Enhance security and streamline entry/exit process
3. Mobile App and Online Booking
    - Develop a mobile app for users to book parking slots in advance
    - Integrate with online payment gateways for seamless transactions
4. Dynamic Pricing
    - Implement dynamic pricing based on demand and availability
    - Offer discounts for off-peak hours or loyalty rewards
5. Data Analytics and Reporting
    - Generate reports on parking trends, occupancy rates, and revenue
    - Use data insights to optimize parking operations and improve user experience
6. Integration with Navigation Systems
    - Integrate with navigation apps (e.g., Google Maps, Waze) for turn-by-turn directions to available parking slots
    - Reduce congestion and improve user experience
7. Automated Payment and Exit
    - Implement automated payment systems (e.g., RFID, NFC)
    - Enable seamless exit without manual intervention

REFERENCES

1. Lipschutz, Seymour. Data Structures with C. McGraw Hill Education, 2017.

2. Tanenbaum, Andrew S., and Langsam, Yedidyah. Data Structures Using C and C++. Pearson Education, 2019.

3. Goodrich, Michael, and Tamassia, Roberto. Data Structures and Algorithms in Java. Wiley Publications, 2018.

4. GeeksforGeeks. “Stack Data Structure – Applications and Implementation.” GeeksforGeeks Online Learning Platform, 2024.

5. Javatpoint. “Stack in Data Structure.” Javatpoint Tutorials, 2024.

6. Weiss, Mark Allen. Data Structures and Algorithm Analysis in C. Addison-Wesley, 2016.

7. R. Sedgewick and K. Wayne. Algorithms. Addison-Wesley Professional, 4th Edition, 2017.

8. IEEE Papers – “Smart Parking Systems and Structural Designs Using Data Structures.” IEEE Access Journal, 2022.

9. Tutorialspoint Online. “Stack – Data Structure Basics, Characteristics, and Applications.” Tutorialspoint, 2024.

10. Cormen, Thomas H., Leiserson, Charles E., Rivest, Ronald L., and Stein, Clifford. Introduction to Algorithms. MIT Press, 3rd Edition, 2022.